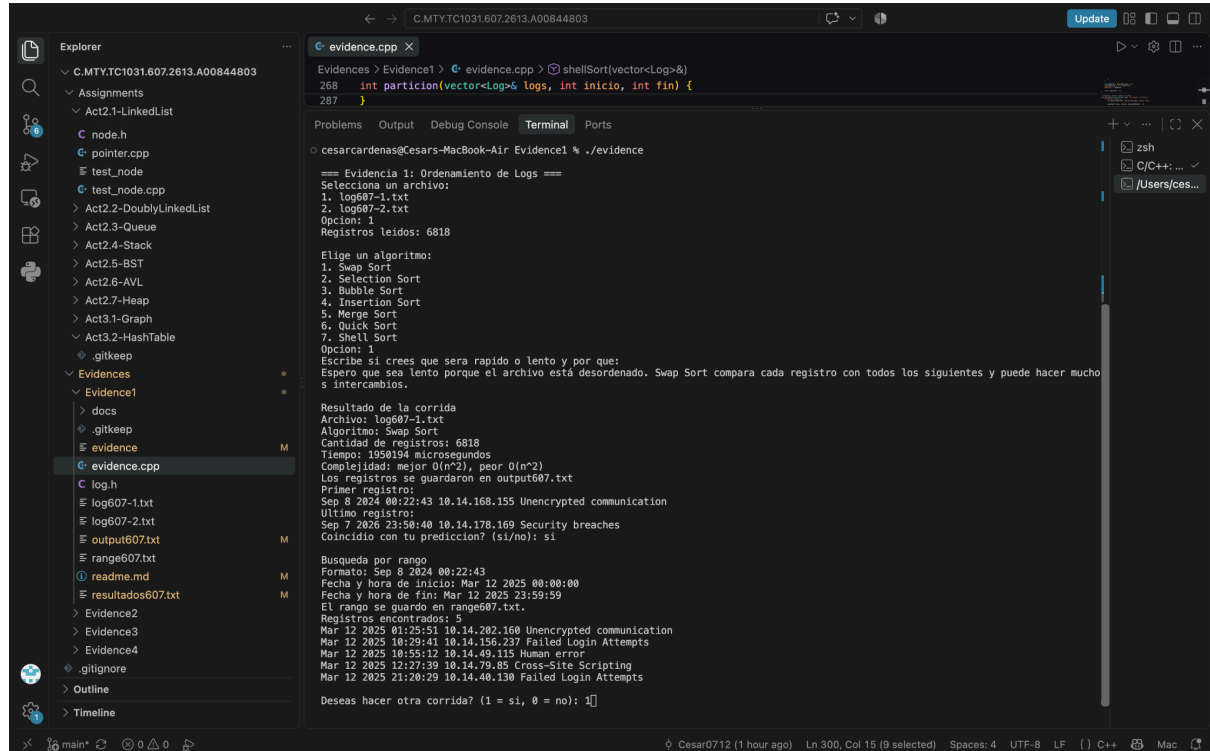


Cesar Aldahir Cardenas Navarro - A00844803

Pruebas con log607-1.txt

Algoritmo Swap Sort



```

C.MTY.TC1031.607.2613.A00844803

evidence.cpp
Evidences > Evidence1 > C: evidence.cpp > shellSort(vector<Log>&)
268 int particion(vector<Log>& logs, int inicio, int fin) {
287 }

Problems Output Debug Console Terminal Ports
cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 1
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 1

Escribe si crees que sera rapido o lento y por que:
Espero que sea lento porque el archivo está desordenado. Swap Sort compara cada registro con todos los siguientes y puede hacer muchos intercambios.

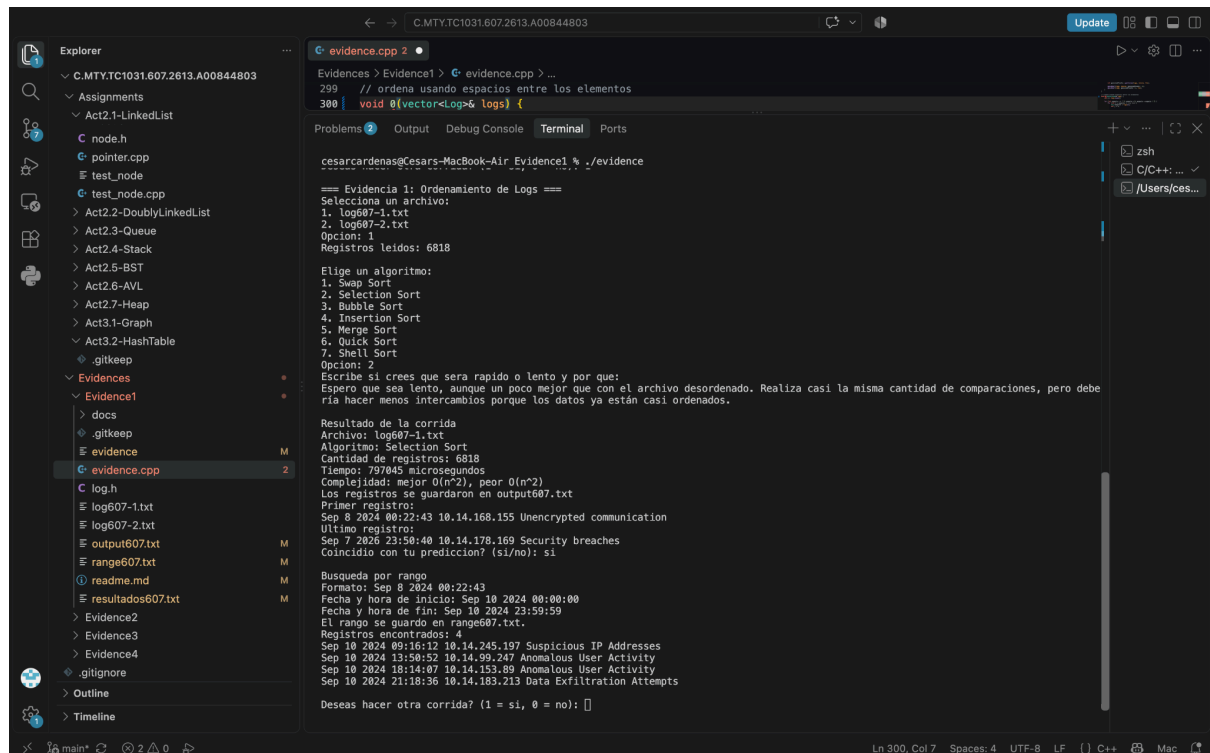
Resultado de la corrida
Archivo: log607-1.txt
Algoritmo: Swap Sort
Cantidad de registros: 6818
Tiempo: 1950194 microsegundos
Complejidad: mejor O(n^2), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:58:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Mar 12 2025 00:00:00
Fecha y hora de fin: Mar 12 2025 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 5
Mar 12 2025 01:25:51 10.14.202.160 Unencrypted communication
Mar 12 2025 18:29:41 10.14.156.237 Failed Login Attempts
Mar 12 2025 16:55:12 10.14.49.115 Human error
Mar 12 2025 12:27:39 10.14.79.85 Cross-Site Scripting
Mar 12 2025 21:20:29 10.14.40.130 Failed Login Attempts

Deseas hacer otra corrida? (1 = si, 0 = no): 1

```

Algoritmo Selection Sort



```

C.MTY.TC1031.607.2613.A00844803

evidence.cpp 2
Evidences > Evidence1 > C: evidence.cpp > ...
299 // ordena usando espacios entre los elementos
300 void q(vector<Log>& logs) {

Problems Output Debug Console Terminal Ports
cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 1
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 2

Escribe si crees que sera rapido o lento y por que:
Espero que sea lento, aunque un poco mejor que con el archivo desordenado. Realiza casi la misma cantidad de comparaciones, pero debería hacer menos intercambios porque los datos ya están casi ordenados.

Resultado de la corrida
Archivo: log607-1.txt
Algoritmo: Selection Sort
Cantidad de registros: 6818
Tiempo: 797045 microsegundos
Complejidad: mejor O(n^2), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:58:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Sep 10 2024 00:00:00
Fecha y hora de fin: Sep 10 2024 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 4
Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses
Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity
Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
Sep 10 2024 21:18:36 10.14.163.213 Data Exfiltration Attempts

Deseas hacer otra corrida? (1 = si, 0 = no): 0

```

Algoritmo Bubble Sort

```
299 // ordena usando espacios entre los elementos
300 void ordena_usando_espacios(vector<Log> & logs) {

cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence ...
Sep 10 2024 13:50:57 10.14.99.247 Anomalous User Activity
Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
Sep 10 2024 21:18:36 10.14.183.213 Data Exfiltration Attempts

Deseas hacer otra corrida? (1 = si, 0 = no): 1

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 1
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 3
Escribe si crees que sera rapido o lento y por que:
Espero que sea lento porque debe realizar muchas pasadas e intercambios para acomodar un archivo desordenado. Su complejidad es O(n^2)

Resultado de la corrida
Archivo: log607-1.txt
Algoritmo: Bubble Sort
Cantidad de registros: 6818
Tiempo: 1914751 microsegundos
Complejidad: mejor O(n^2), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidio con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: 16 Dic 2025 00:00:00
Fecha y hora de fin: Dic 16 2025 23:59:59
El formato de alguna fecha no es valido.

Deseas hacer otra corrida? (1 = si, 0 = no): 1
```

Algoritmo Insertion Sort

```
299 // ordena usando espacios entre los elementos
300 void ordena_usando_espacios(vector<Log> & logs) {

cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence ...
Fecha y hora de inicio: 16 Dic 2025 00:00:00
Fecha y hora de fin: Dic 16 2025 23:59:59
El formato de alguna fecha no es valido.

Deseas hacer otra corrida? (1 = si, 0 = no): 1

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 1
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 4
Escribe si crees que sera rapido o lento y por que:
Espero que sea lento porque tendra que mover muchos registros para insertarlos en su posicion correcta. En datos desordenados puede l
legar a O(n^2).

Resultado de la corrida
Archivo: log607-1.txt
Algoritmo: Insertion Sort
Cantidad de registros: 6818
Tiempo: 702509 microsegundos
Complejidad: mejor O(n), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidio con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Dic 25 2024 00:00:00
Fecha y hora de fin: Dic 25 2024 23:59:59
El formato de alguna fecha no es valido.

Deseas hacer otra corrida? (1 = si, 0 = no): 1
```

Algoritmo Merge Sort

The screenshot shows a code editor with a file explorer on the left and a terminal on the right. The file explorer shows a project structure with files like `evidence.cpp`, `log.h`, and `log607-1.txt`. The terminal displays the output of running the program, which includes a menu of sorting algorithms, the selection of Merge Sort, and the execution of the algorithm on a log file. The output shows the number of records (6818), the time taken (77455 microseconds), and the complexity (O(n log n)). It also displays a list of security breaches from a log file, including port scanning, unauthorized access, and failed login attempts.

```
evidences > Evidence1 > C:\evidence.cpp > ...
299 // ordena usando espacios entre los elementos
300 void 0(vector<log> logs) {

cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence
2. log607-2.txt
Option: 1
Registros leídos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Option: 5
Escribe si crees que sera rapido o lento y por que:
Espero que sea rápido porque divide los datos en partes pequeñas y su complejidad es O(n log n), incluso cuando el archivo está desordenado.

Resultado de la corrida
Archivo: log607-1.txt
Algoritmo: Merge Sort
Cantidad de registros: 6818
Tiempo: 77455 microsegundos
Complejidad: mejor O(n log n), peor O(n log n)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: May 02 2025 00:00:00
Fecha y hora de fin: May 02 2025 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 0
May 2 2025 00:57:16 10.14.69.143 Port Scanning Activity
May 2 2025 02:24:54 10.14.148.59 Unauthorized Access Attempts
May 2 2025 02:45:31 10.14.19.47 Data Exfiltration Attempts
May 2 2025 03:16:18 10.14.117.6 Port Scanning Activity
May 2 2025 09:40:32 10.14.118.136 Encryption Failures
May 2 2025 18:38:02 10.14.177.219 Failed Login Attempts
May 2 2025 21:32:46 10.14.168.124 Cross-Site Scripting
May 2 2025 23:52:33 10.14.170.87 Unauthorized Access Attempts

Deseas hacer otra corrida? (1 = si, 0 = no):
```

Algoritmo Quick Sort

The screenshot shows a Visual Studio Code editor with a C++ project. The Explorer panel on the left shows a file tree with folders like 'Assignments', 'Evidence1', and 'Evidence2'. The main editor displays the 'evidence.cpp' file, which contains a Quick Sort implementation. The output window shows the execution results, including the selection of the Quick Sort algorithm, the number of records (6818), and the execution time (25823 microseconds). The output also shows the first and last records of the sorted data.

```
299 // ordena usando espacios entre los elementos
300 void qsort(vector<Log> & logs) {

cescardenas@Cesars-MacBook-Air Evidence1 % ./evidence
=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 1
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 6
Escribe si crees que sera rapido o lento y por que:
Espero que sea rapido porque en datos desordenados el pivot puede producir divisiones relativamente equilibradas. Su caso promedio es O(n log n).

Resultado de la corrida
Archivo: log607-1.txt
Algoritmo: Quick Sort
Cantidad de registros: 6818
Tiempo: 25823 microsegundos
Complejidad: mejor O(n log n), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:58:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Mar 12 2025 00:00:00
Fecha y hora de fin: Mar 12 2025 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 5
Mar 12 2025 01:25:51 10.14.202.160 Unencrypted communication
Mar 12 2025 10:29:41 10.14.156.237 Failed Login Attempts
Mar 12 2025 18:55:12 10.14.49.115 Human error
Mar 12 2025 12:27:39 10.14.79.85 Cross-Site Scripting
Mar 12 2025 21:20:29 10.14.40.130 Failed Login Attempts

Deseas hacer otra corrida? (1 = si, 0 = no):
```

Algoritmo Shell Sort

The screenshot shows a Visual Studio Code editor with a C++ project. The Explorer panel on the left shows a file tree with folders like 'Assignments', 'Evidence1', and 'Evidence2'. The main editor displays the 'evidence.cpp' file, which contains a Shell Sort implementation. The output window shows the execution results, including the selection of the Shell Sort algorithm, the number of records (6818), and the execution time (24368 microseconds). The output also shows the first and last records of the sorted data.

```
299 // ordena usando espacios entre los elementos
300 void qsort(vector<Log> & logs) {

cescardenas@Cesars-MacBook-Air Evidence1 % ./evidence
Mar 12 2025 10:55:12 10.14.49.115 Human error
Mar 12 2025 12:27:39 10.14.79.85 Cross-Site Scripting
Mar 12 2025 21:20:29 10.14.40.130 Failed Login Attempts

Deseas hacer otra corrida? (1 = si, 0 = no): 1

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 1
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 7
Escribe si crees que sera rapido o lento y por que:
Espero que tenga una velocidad media o rapida porque primero mueve registros usando espacios grandes, reduciendo el desorden antes de hacer comparaciones cercanas.

Resultado de la corrida
Archivo: log607-1.txt
Algoritmo: Shell Sort
Cantidad de registros: 6818
Tiempo: 24368 microsegundos
Complejidad: mejor O(n log n), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:58:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Abr 1 2026 00:00:00
Fecha y hora de fin: Abr 1 23:59:59
El formato de alguna fecha no es valido.

Deseas hacer otra corrida? (1 = si, 0 = no):
```

Pruebas con log607-2.txt

Algoritmo Swap Sort

```
evidence.cpp 2
Evidences > Evidence1 > C: evidence.cpp > ...
299 // ordena usando espacios entre los elementos
300 { void d(vector<Log>& logs) {

Problems Output Debug Console Terminal Ports
cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 2
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 1
Escribe si crees que sera rapido o lento y por que:
Espero que sea lento, aunque un poco mejor que con el archivo desordenado. Realiza casi la misma cantidad de comparaciones, pero deberia hacer menos intercambios porque los datos ya están casi ordenados.

Resultado de la corrida
Archivo: log607-2.txt
Algoritmo: Swap Sort
Cantidad de registros: 6818
Tiempo: 812121 microsegundos
Complejidad: mejor O(n^2), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Ene 05 2025 00:00:00
Fecha y hora de fin: Ene 05 2025 23:59:59
El formato de alguna fecha no es valido.

Deseas hacer otra corrida? (1 = si, 0 = no): []
```

Algoritmo Selection Sort

```
evidence.cpp 2
Evidences > Evidence1 > C: evidence.cpp > ...
299 // ordena usando espacios entre los elementos
300 { void d(vector<Log>& logs) {

Problems Output Debug Console Terminal Ports
cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 2
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 2
Escribe si crees que sera rapido o lento y por que:
Espero que siga siendo lento porque Selection Sort hace prácticamente la misma cantidad de comparaciones aunque el archivo ya esté ca si ordenado.

Resultado de la corrida
Archivo: log607-2.txt
Algoritmo: Selection Sort
Cantidad de registros: 6818
Tiempo: 797644 microsegundos
Complejidad: mejor O(n^2), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Mar 12 2025 00:00:00
Fecha y hora de fin: Mar 12 2025 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 5
Mar 12 2025 01:25:51 10.14.202.160 Unencrypted communication
Mar 12 2025 10:20:41 10.14.156.237 Failed Login Attempts
Mar 12 2025 10:55:12 10.14.49.115 Human error
Mar 12 2025 12:27:39 10.14.79.85 Cross-Site Scripting
Mar 12 2025 21:20:29 10.14.40.130 Failed Login Attempts

Deseas hacer otra corrida? (1 = si, 0 = no): []
```

Algoritmo Bubble Sort

```

C.MTY.TC1031.607.2613.A00844803
Assignments
  Act2.1-LinkedList
  C node.h
  pointer.cpp
  test_node
  test_node.cpp
  Act2.2-DoublyLinkedList
  Act2.3-Queue
  Act2.4-Stack
  Act2.5-BST
  Act2.6-AVL
  Act2.7-Heap
  Act3.1-Graph
  Act3.2-HashTable
  .gitkeep
  Evidence1
    docs
    .gitkeep
    evidence
    evidence.cpp
    log.h
    log607-1.txt
    log607-2.txt
    output607.txt
    range607.txt
    readme.md
    resultados607.txt
  Evidence2
  Evidence3
  Evidence4
  .gitignore
  Outline
  Timeline

C.MTY.TC1031.607.2613.A00844803
evidence.cpp 2
// ordena usando espacios entre los elementos
void @vector<log> logs) {

cescardenas@Cesars-MacBook-Air Evidence1 % ./evidence

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 2
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 3
Escribe si crees que sera rapido o lento y por que:
Espero que sea lento, pero con menos intercambios. Mi version no tiene una salida anticipada, asi que todavia realiza todas las pasadas aunque los datos esten casi ordenados.

Resultado de la corrida
Archivo: log607-2.txt
Algoritmo: Bubble Sort
Cantidad de registros: 6818
Tiempo: 818040 microsegundos
Complejidad: mejor O(n^2), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:58:40 10.14.178.169 Security breaches
Coincidio con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Sep 02 2025 00:00:00
Fecha y hora de fin: Sep 02 2025 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 4
Sep 2 2025 01:53:23 10.14.203.226 Application error
Sep 2 2025 13:47:15 10.14.88.86 Unauthorized Access Attempts
Sep 2 2025 16:56:36 10.14.105.7 Broken authentication and session management
Sep 2 2025 17:25:21 10.14.109.44 Load balancing issues

Deseas hacer otra corrida? (1 = si, 0 = no): 0
Ln 300, Col 7 Spaces: 4 UTF-8 LF C++ Mac
```

Algoritmo Insertion Sort

```

C.MTY.TC1031.607.2613.A00844803
Assignments
  Act2.1-LinkedList
  C node.h
  pointer.cpp
  test_node
  test_node.cpp
  Act2.2-DoublyLinkedList
  Act2.3-Queue
  Act2.4-Stack
  Act2.5-BST
  Act2.6-AVL
  Act2.7-Heap
  Act3.1-Graph
  Act3.2-HashTable
  .gitkeep
  Evidence1
    docs
    .gitkeep
    evidence
    evidence.cpp
    log.h
    log607-1.txt
    log607-2.txt
    output607.txt
    range607.txt
    readme.md
    resultados607.txt
  Evidence2
  Evidence3
  Evidence4
  .gitignore
  Outline
  Timeline

C.MTY.TC1031.607.2613.A00844803
evidence.cpp 2
// ordena usando espacios entre los elementos
void @vector<log> logs) {

cescardenas@Cesars-MacBook-Air Evidence1 % ./evidence

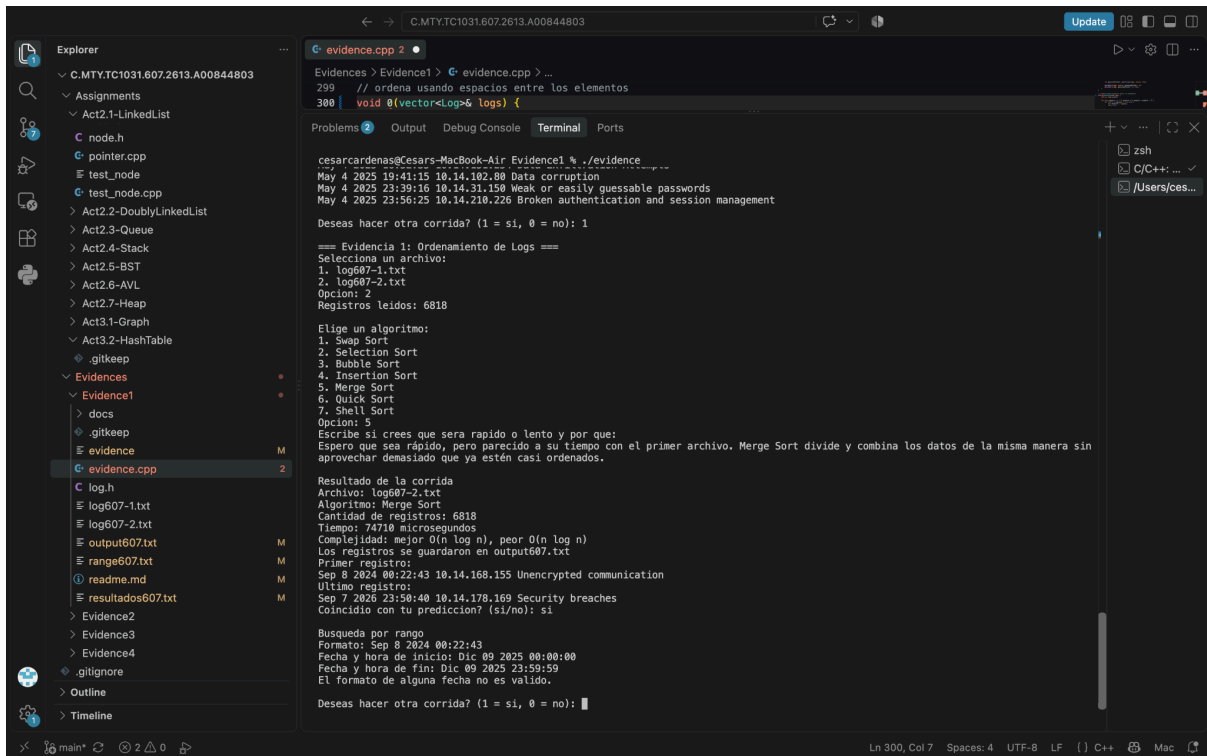
Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 4
Escribe si crees que sera rapido o lento y por que:
Espero que sea muy rapido porque los registros ya estan cerca de su posicion final. Insertion Sort normalmente necesita pocos movimientos con datos casi ordenados.

Resultado de la corrida
Archivo: log607-2.txt
Algoritmo: Insertion Sort
Cantidad de registros: 6818
Tiempo: 7900 microsegundos
Complejidad: mejor O(n), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:58:40 10.14.178.169 Security breaches
Coincidio con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: May 04 2025 00:00:00
Fecha y hora de fin: May 04 2025 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 12
May 4 2025 06:02:09 10.14.49.136 Failed Login Attempts
May 4 2025 06:50:35 10.14.220.66 Broken authentication and session management
May 4 2025 07:53:39 10.14.102.151 Social engineering attacks
May 4 2025 13:29:23 10.14.32.194 Network connectivity issues
May 4 2025 14:12:50 10.14.148.211 Human error
May 4 2025 14:32:11 10.14.187.10 Load balancing issues
May 4 2025 15:41:58 10.14.32.229 Load balancing issues
May 4 2025 15:46:52 10.14.16.232 Weak or easily guessable passwords
May 4 2025 16:52:35 10.14.151.254 Data Exfiltration Attempts
May 4 2025 19:41:15 10.14.102.80 Data corruption
May 4 2025 23:39:16 10.14.31.150 Weak or easily guessable passwords
May 4 2025 23:56:25 10.14.210.226 Broken authentication and session management

Deseas hacer otra corrida? (1 = si, 0 = no): 0
Ln 300, Col 7 Spaces: 4 UTF-8 LF C++ Mac
```

Algoritmo Merge Sort



```

C:\MTY.TC1031.607.2613.A00844803
C:\evidence.cpp 2
Evidences > Evidence1 > C:\evidence.cpp > ...
299 // ordena usando espacios entre los elementos
300 void 0(vector<Log*> logs) {

Problems Output Debug Console Terminal Ports
cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence
May 4 2025 19:41:15 10.14.102.80 Data corruption
May 4 2025 23:39:16 10.14.31.150 Weak or easily guessable passwords
May 4 2025 23:56:25 10.14.210.226 Broken authentication and session management

Deseas hacer otra corrida? (1 = si, 0 = no): 1

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 2
Registros leidos: 6818

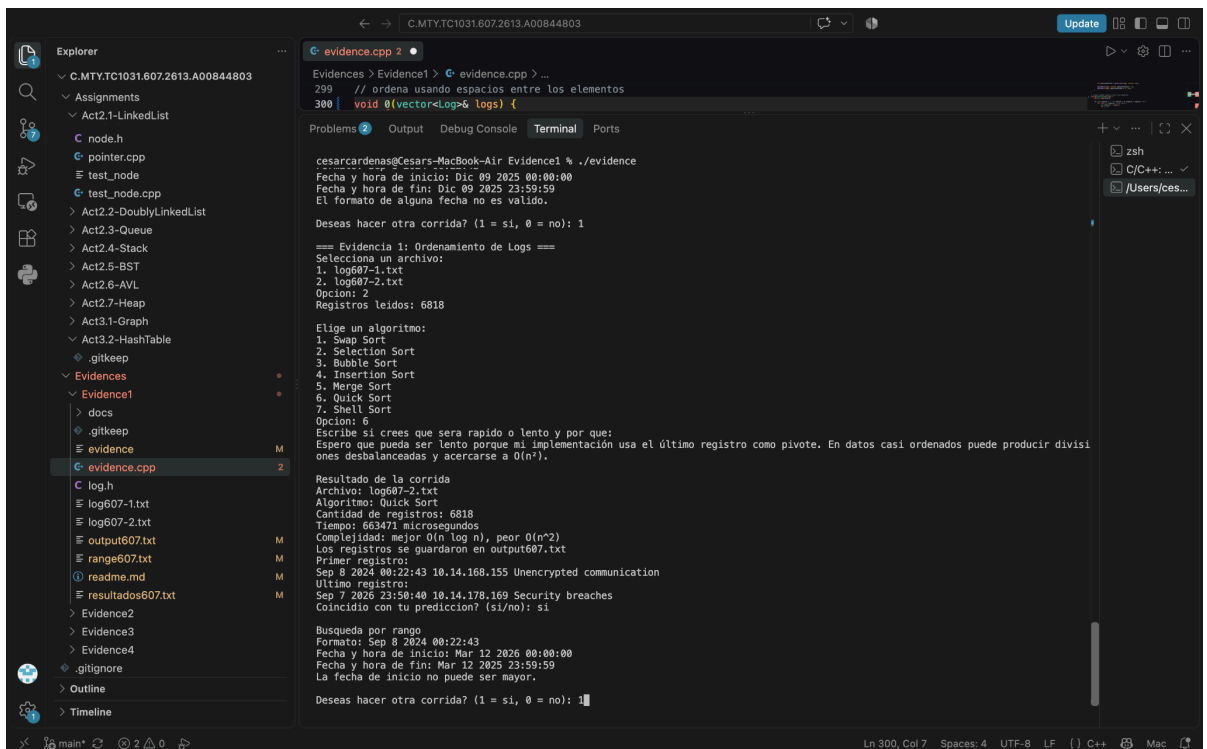
Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 5
Escribe si crees que sera rapido o lento y por que:
Espero que sea rapido, pero parecido a su tiempo con el primer archivo. Merge Sort divide y combina los datos de la misma manera sin
aprovechar demasiado que ya estén casi ordenados.

Resultado de la corrida
Archivo: log607-2.txt
Algoritmo: Merge Sort
Cantidad de registros: 6818
Tiempo: 74710 microsegundos
Complejidad: mejor O(n log n), peor O(n log n)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidio con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Dic 09 2025 00:00:00
Fecha y hora de fin: Dic 09 2025 23:59:59
El formato de alguna fecha no es valido.

Deseas hacer otra corrida? (1 = si, 0 = no): 1
Ln 300, Col 7 Spaces: 4 UTF-8 LF C++ Mac
```

Algoritmo Quick Sort



```

C:\MTY.TC1031.607.2613.A00844803
C:\evidence.cpp 2
Evidences > Evidence1 > C:\evidence.cpp > ...
299 // ordena usando espacios entre los elementos
300 void 0(vector<Log*> logs) {

Problems Output Debug Console Terminal Ports
cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence
Fecha y hora de inicio: Dic 09 2025 00:00:00
Fecha y hora de fin: Dic 09 2025 23:59:59
El formato de alguna fecha no es valido.

Deseas hacer otra corrida? (1 = si, 0 = no): 1

=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 2
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 6
Escribe si crees que sera rapido o lento y por que:
Espero que pueda ser lento porque mi implementación usa el último registro como pivote. En datos casi ordenados puede producir divisi
ones desbalanceadas y acercarse a O(n^2).

Resultado de la corrida
Archivo: log607-2.txt
Algoritmo: Quick Sort
Cantidad de registros: 6818
Tiempo: 663471 microsegundos
Complejidad: mejor O(n log n), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidio con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Mar 12 2026 00:00:00
Fecha y hora de fin: Mar 12 2025 23:59:59
La fecha de inicio no puede ser mayor.

Deseas hacer otra corrida? (1 = si, 0 = no): 1
Ln 300, Col 7 Spaces: 4 UTF-8 LF C++ Mac
```

Algoritmo Shell Sort

```

C:\MTY.TC1031.607.2613.A00844803
C:\evidence.cpp 2
Evidencias > Evidence1 > C:\evidence.cpp > ...
299 // ordena usando espacios entre los elementos
300 void <vector<log*> logs) {

Problems Output Debug Console Terminal Ports
cesarcardenas@Cesars-MacBook-Air Evidence1 % ./evidence
=== Evidencia 1: Ordenamiento de Logs ===
Selecciona un archivo:
1. log607-1.txt
2. log607-2.txt
Opcion: 2
Registros leidos: 6818

Elige un algoritmo:
1. Swap Sort
2. Selection Sort
3. Bubble Sort
4. Insertion Sort
5. Merge Sort
6. Quick Sort
7. Shell Sort
Opcion: 7
Escribe si crees que sera rapido o lento y por que:
Espero que sea rapido porque el archivo ya esta casi ordenado y Shell Sort tendra que hacer pocos movimientos para terminar de acomodarlo.

Resultado de la corrida
Archivo: log607-2.txt
Algoritmo: Shell Sort
Cantidad de registros: 6818
Tiempo: 16798 microsegundos
Complejidad: mejor O(n log n), peor O(n^2)
Los registros se guardaron en output607.txt
Primer registro:
Sep 8 2024 00:22:43 10.14.168.155 Unencrypted communication
Ultimo registro:
Sep 7 2026 23:50:40 10.14.178.169 Security breaches
Coincidió con tu prediccion? (si/no): si

Busqueda por rango
Formato: Sep 8 2024 00:22:43
Fecha y hora de inicio: Mar 12 2025 00:00:00
Fecha y hora de fin: Mar 12 2025 23:59:59
El rango se guardo en range607.txt.
Registros encontrados: 5
Mar 12 2025 01:25:51 10.14.202.160 Unencrypted communication
Mar 12 2025 10:29:41 10.14.156.237 Failed Login Attempts
Mar 12 2025 10:55:12 10.14.49.115 Human error
Mar 12 2025 12:27:39 10.14.79.85 Cross-Site Scripting
Mar 12 2025 21:20:29 10.14.40.130 Failed Login Attempts

Deseas hacer otra corrida? (1 = si, 0 = no):
```